



1 Classe para representar um ponto no plano cartesiano

Crie uma classe chamada `Ponto` que represente um ponto no plano cartesiano (x , y). A classe deve ter os seguintes métodos:

- Método construtor que receba como parâmetros os valores de x e y ;
- Métodos modificadores (*set*) para alterar os valores de cada um dos atributos;
- Métodos acessores (*get*) para retornar os valores de cada um dos atributos;
- Método `toString()` para retornar o ponto no formato (x , y);
- Método que receba como parâmetro um objeto da classe `Ponto` e retorne a distância entre os dois pontos.

– A fórmula para calcular a distância entre dois pontos é: $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

2 Classe para apresentar um horário

Crie uma classe chamada `Horario` para representar um horário (hora, minuto e segundo). A classe deverá ter os seguintes métodos:

- Método construtor padrão que inicialize todos atributos com zero;
- Métodos construtores sobrecarregados que permitam inicializar os atributos com valores fornecidos como argumentos para: i) hora; ii) hora e minuto; e iii) hora, minuto e segundo. Caso algum dos valores não seja válido, o método construtor deve inicializar todos os atributos com zero;
- Métodos modificadores (*set*) para alterar os valores de cada um dos atributos. Caso o valor fornecido não seja válido, o método deve manter o valor atual do atributo e retornar `false`. Caso contrário, o método deve alterar o valor do atributo e retornar `true`;
- Método `toString()` para retornar o horário no formato HH:MM:SS (por exemplo, 23:59:59);
- Método para imprimir por extenso o horário. Por exemplo, *"vinte e três horas, cinquenta e nove minutos e cinquenta e nove segundos"*;
- Método que retorne o horário em segundos (um valor do tipo `long`). Por exemplo, se horário for 01:00:00, então deve retornar 3600;
- Método que receba como parâmetro um objeto da classe `Horario` e retorne a diferença em segundos entre os dois horários.

3 Classe para representar um retângulo

Crie uma classe chamada `Retangulo` para representar um retângulo. A classe deverá ter os seguintes métodos:

- Método construtor que receba como parâmetros, a largura, a altura do retângulo e a codificação de caracteres para desenhar o retângulo (uma `String` com ASCII ou UTF8). Caso a largura ou a altura fornecida seja menor ou igual a zero, o método construtor deve inicializar os atributos com valores padrão (largura = 4, altura = 3). Caso a codificação de caracteres fornecida seja diferente de ASCII ou UTF8, o método construtor deve inicializar o atributo com o valor padrão (codificação = ASCII);
- Método construtor padrão que inicialize todos atributos com valores padrão (largura = 4, altura = 3, codificação = ASCII);
- Métodos modificadores (*set*) para alterar os valores de cada um dos atributos. Caso o valor fornecido não seja válido, o método deve manter o valor atual do atributo e retornar `false`. Caso contrário, o método deve alterar o valor do atributo e retornar `true`;
- Métodos acessores (*get*) para retornar os valores de cada um dos atributos;
- Método que retorne a área do retângulo. A área de um retângulo é dada por: $A = \text{largura} \times \text{altura}$;
- Método que retorne o perímetro do retângulo. O perímetro de um retângulo é dado por: $P = 2 \times (\text{largura} + \text{altura})$;
- Método `toString()` para retornar uma representação do retângulo no formato de uma *string*. Por exemplo, para um retângulo de largura 4 e altura 3 e com codificação em ASCII a representação seria:

```
+---+
|   |
+---+
```

- Para um retângulo de largura 4 e altura 3 e com codificação em UTF8 a representação seria (`String` em Java)¹:

```
String r = "\u250c\u2500\u2500\u2500\u2510\u2502 \u2502\u2502\u2502\u2500\u2500\u2500\u2518";
System.out.println(r);
```

4 Classe para representar uma Data

Crie uma classe chamada `Data` para representar uma data (dia, mês e ano). A classe deverá ter os seguintes métodos:

- Método construtor padrão que inicialize com a data 01/01/1970;
- Métodos construtores sobrecarregados que permitam inicializar os atributos com valores fornecidos como argumentos para: i) dia; ii) dia e mês; e iii) dia, mês e ano. Caso algum dos valores não seja válido, o método construtor deve inicializar todos os atributos com valores padrão (01/01/1970);
 - A classe deve garantir que a data fornecida seja válida. Por exemplo, a data 31/02/2024 não é válida, pois em 2024 o mês de fevereiro tem no máximo 29 dias.
 - Assim, a classe deverá ter ciência de quantos dias tem cada mês e se o ano é bissexto ou não. Um ano é bissexto se for divisível por 4, exceto os anos que são divisíveis por 100 e não são divisíveis por 400.

¹https://en.wikipedia.org/wiki/Box-drawing_character

- Métodos modificadores (*set*) para alterar os valores de cada um dos atributos. Caso o valor fornecido não seja válido, o método deve manter o valor atual do atributo e retornar *false*. Caso contrário, o método deve alterar o valor do atributo e retornar *true*;
- Métodos acessores (*get*) para retornar os valores de cada um dos atributos;
- Método *toString()* para retornar a data no formato DD/MM/AAAA (por exemplo, 31/12/2024);
- Método que retorne a data por extenso. Por exemplo, "31 de dezembro de 2024";
- Método que receba como parâmetro um objeto da classe *Data* e retorne a diferença em dias entre as duas datas.

5 Classe para representar um robô

Todo robô deve ter um nome (uma *string*) e um número de série (um número inteiro). O número de série deve ser gerado automaticamente a partir de um contador que é incrementado a cada vez que um novo objeto da classe *Robo* é criado. Todo objeto da classe *Robo* deve ter ciência de quantos objetos foram criados. Além disso, o número de série de um robô não pode ser alterado depois que ele é criado. Por fim, pense em quais atributos e métodos podem ser úteis para a classe *Robo* e implemente-os.